

Interview Script — Senior Snowflake Data Engineer

Candidate: Abdul | 12 Years Total Exp | 4+ Years Snowflake | Project: BHP (Oil & Gas, Australia) via CTS

1. Opening — "Tell Me About Yourself"

Q: Walk me through your background and current role.

Model Answer (personalize with your own words): "I have 12 years of overall IT experience. I started as a SQL Developer at Mouritech, then moved to Capgemini where I worked across a few different technologies, and for the last 4+ years I've been working as a Senior Snowflake Data Engineer at CTS, on a client project for BHP — a major Oil & Gas company based in Australia.

In this project, my core stack is **Snowflake, DBT, Airflow (via Astronomer), and CI/CD**. We integrate SAP ERP data — Sales, Procurement, Warehouse & Distribution, Finance, and SuccessFactors HR data — along with some API data, into a Snowflake-based enterprise data warehouse. We follow a **Medallion architecture (Bronze/RAW → Silver/QA → Gold/Conformed)**, and I'm involved end-to-end — from framework-based pipeline generation, DBT modeling, unit testing, UAT, to CI/CD deployment into PROD and post-go-live support handover."

Follow-up interviewers often ask immediately: **"What exactly was your individual contribution vs. the team's?"** → Be ready with specifics: which SAP modules/t-codes you personally built pipelines for, how many pipelines/models you've built, any performance issue you personally fixed, any migration or optimization you led.

2. Project Deep-Dive Questions

Q1: You mentioned SAP data comes in via a Datavard team. Explain the full data flow from SAP to your Snowflake Gold layer.

Expected answer structure:

1. Datavard team extracts SAP data (via SAP t-codes / tables) and drops **2 files** into AWS S3: a schema file (`%s_.csv`) and a data file (header + sample data).
2. Your in-house **Python framework** reads the schema file and auto-generates: dataset config, pipeline file, and Airflow DAG file — no manual table DDL needed.

3. Framework converts CSV → **Parquet** and loads into **Bronze/RAW** (technical SAP column names, e.g., raw EBAN, LFA1, VBAP, VBAK) using `COPY INTO`.
4. **Silver/QA layer**: technical names mapped to meaningful business names.
5. **Gold/Conformed layer**: built in **DBT** — business transformations (e.g., total price calc from VBAP+VBAK for a t-code like KS03), tagging, tests.
6. Code pushed to Git → PR merge → DAG triggered in Airflow via **Astronomer** → Unit testing → UAT → CI/CD to QA/PROD.

Be ready for: "Why Parquet and not just load CSV directly?" → columnar format, better compression, faster COPY INTO performance, schema enforcement, splitting for parallel load.

Q2: What exactly does your Python framework generate, and why build a framework instead of writing pipelines manually each time?

Talking points: standardization across hundreds of SAP tables/t-codes, faster onboarding of new sources (schema-driven, config-driven), reduces manual DDL/pipeline errors, enforces naming/tagging conventions automatically, easier to scale for a large enterprise with many SAP modules.

Q3: Give an example — how do you handle a specific SAP t-code end-to-end, e.g., KS03?

Use your own example: KS03 pulls from VBAP (sales document item) + VBAK (sales document header) → lands in Bronze as raw → Silver renames columns meaningfully → Gold/DBT model joins VBAP+VBAK to calculate total price, applies business tests and mandatory tags.

Q4: What is Unit Testing in your pipeline context? What exactly do you check?

- Airflow DAG run success screenshot
- Primary key duplicate checks
- Sample data validation against source
- Row count reconciliation (source vs target)

Follow-up: "What automated tests do you write in DBT specifically?" (see DBT section below — be ready to name `unique`, `not_null`, `relationships`, `accepted_values`, custom singular tests).

Q5: Explain your full deployment lifecycle from Dev to PROD.

Dev → Unit Test → Client review/UAT sign-off → CI/CD push to QA → **Change Request (CR)** raised for PROD (attaching UAT sign-off email, KT session recording, Data Owner approval email) → MR/PR merge to PROD → **KT session to Ops/Support team** before go-live → Deploy to PROD → Update **Confluence** documentation → **Hypercare/monitoring for 5-7 days** → Handover ownership to Ops team.

Follow-up: "Why is a Change Request important in an enterprise setting?" → governance, audit trail, risk control, rollback plan, accountability — common in regulated industries like Oil & Gas.

Q6: What's the difference between RAW, QA, and Conformed layers in your project, and why does this matter?

RAW = technical/source-faithful (audit, reprocessing). QA = business-readable, still granular. Conformed = business-ready, tested, DBT-modeled, consumed by BI. This is essentially Bronze/Silver/Gold — be ready to map it to standard Medallion terminology since interviewers may use either naming.

3. DBT-Specific Questions

Q1: What role does DBT play in your architecture — where does it sit?

DBT sits between Silver/QA and Gold/Conformed. It handles business transformations, joins, calculated fields (e.g., total price), documentation, tagging, and testing — not ingestion.

Q2: What kind of DBT models have you built? (staging/intermediate/mart?)

Be ready to describe your model layering: staging models (1:1 with source, light cleanup) → intermediate (joins, e.g., VBAP+VBAK) → mart/final models (business-ready, e.g., Sales Fact, Procurement Fact).

Q3: What tests do you write in DBT and why?

- `unique` / `not_null` on PKs
- `relationships` (referential integrity between fact/dim)
- `accepted_values` for classification/status columns
- Custom singular tests for business rules (e.g., `total_price` should never be negative)

Q4: How do you schedule and monitor DBT jobs?

Via DBT job scheduling (or orchestrated through Airflow tasks), with alerting on test failures before data reaches Conformed/Gold, which downstream Power BI dashboards depend on.

Q5 (Deeper, likely asked): "What's the difference between a DBT `view`, `table`, and `incremental` materialization? Which do you use where and why?" → Be ready — this is commonly asked and not explicitly in your notes. Views for lightweight staging, tables for stable marts, incremental for large fact tables to avoid full rebuilds (using `is_incremental()` and a unique key strategy — ties directly into your delta load story).

4. Airflow / Orchestration / CI-CD Questions

Q1: Why Airflow, and what does Astronomer add on top of it?

Airflow = DAG-based orchestration for dependencies between ingestion, transformation, DBT runs. Astronomer = managed Airflow platform — simplifies deployment, environment management, and DAG triggering across your framework-generated DAGs.

Q2: How are your DAGs generated — manually written or automated?

Auto-generated by your Python framework based on the schema file — reinforces your framework story, shows scale thinking.

Q3: Walk me through your CI/CD pipeline branch strategy.

Dev → QA → PROD branches; MR/PR-based promotion; CR-gated PROD deployment; this is standard git-flow — be ready to name the tool (Git/GitHub/GitLab/Bitbucket — clarify which you actually use) and any pipeline tool (Jenkins/Azure DevOps/GitHub Actions) since your notes don't specify — **fill this gap before the interview.**

Q4 (Likely gotcha): "What happens if a DAG fails in PROD? What's your incident/rollback process?"

→ Be ready with a concrete answer: alerting mechanism, who gets paged (Ops after hypercare), rollback via previous stable code version, backfill strategy for missed loads.

5. Data Governance & Security Questions

Q1: How is data classified in your project, and what controls follow from that?

Three tiers: **Internal, Confidential, Highly Confidential**. Each object/table gets a mandatory classification tag. Access to Confidential/Highly Confidential data requires a formal request tied to a **BDC ID**.

Q2: How do you handle PII data specifically?

Access-controlled by default — a user cannot view PII unless explicitly granted access. Enforced via **Dynamic Data Masking policies** and **row/column-level security** in Snowflake.

Q3: Explain how Snowflake's masking policy or row access policy actually works (be ready for a technical deep-dive here).

You should be able to sketch, even verbally:

```
sql
CREATE MASKING POLICY mask_ssn AS (val STRING) RETURNS STRING ->
CASE WHEN CURRENT_ROLE() IN ('PII_VIEWER') THEN val ELSE '***MASKED***' END;

ALTER TABLE employee MODIFY COLUMN ssn SET MASKING POLICY mask_ssn;
```

Mention **RBAC** (roles → grants → warehouses/databases/schemas), and that masking is role-based and centrally governed, not per-user.

Q4: How does this compare to Snowflake's native Data Governance features — tagging, Object

Tagging, Classification? → Even if your project's tagging is custom/BDC-driven, mention you're aware

Snowflake has native **Object Tagging** and **Sensitive Data Classification** — shows breadth beyond just “what my project does.”

6. Delta Load / Data Loading Questions

Q1: How do you implement delta/incremental loads?

Framework converts incoming CSV to Parquet, loads to a staging table via `COPY INTO`, then a **MERGE statement** (handled internally by the framework) applies inserts/updates based on changed records only — avoiding full reloads.

Q2: Can you write a MERGE statement for an upsert scenario? (Be ready — likely asked even though not in your notes)

```
sql
MERGE INTO silver.orders AS tgt
USING staging.orders_delta AS src
ON tgt.order_id = src.order_id
WHEN MATCHED AND src.load_timestamp > tgt.load_timestamp THEN
  UPDATE SET tgt.amount = src.amount, tgt.status = src.status, tgt.load_timestamp = src.load_timestamp
WHEN NOT MATCHED THEN
  INSERT (order_id, amount, status, load_timestamp)
VALUES (src.order_id, src.amount, src.status, src.load_timestamp);
```

Q3: How do you handle late-arriving data or duplicate file drops?

Reference your dedup transformation using `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY load_timestamp DESC)`.

Q4: What's the difference between Snowpipe and your COPY INTO + framework approach? Why did the project choose the latter?

Be ready to reason about this even if not explicit in your notes: Snowpipe is good for continuous, event-driven micro-batch loads (auto-ingest via S3 event notifications); your framework's batch/DAG-triggered COPY INTO fits scheduled SAP file drops better and gives more control over sequencing (schema file → data file → transform → load) and validation before load.

7. SQL / Transformation Deep-Dive (use your own examples — be ready to explain, not just recite)

For each of these, be ready to explain **why**, not just show the syntax:

- 1. Deduplication** — `ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY load_timestamp DESC)`
 - *Why DESC on load_timestamp?* → keep latest version of a record.
- 2. SPLIT_PART** — parsing composite keys like `1244-PROD-BHP-4589` into company code, plant, record type, ID.
 - *Follow-up:* "What if the delimiter count varies?" → mention `SPLIT_PART` returns NULL/empty gracefully if position doesn't exist; discuss validation.
- 3. NULL handling** — `COALESCE` vs `NULLIF`.
 - *Follow-up:* "Difference between COALESCE and IFNULL/NVL in Snowflake?" → COALESCE handles multiple args, is ANSI standard; NVL/IFNULL are 2-arg only.
- 4. Type casting** — `TO_DATE`, `TRY_CAST`.
 - *Follow-up:* "Why TRY_CAST instead of CAST?" → TRY_CAST returns NULL on failure instead of erroring out the whole query — critical for messy SAP source data.
- 5. Semi-structured JSON flattening** — `LATERAL FLATTEN` on VARIANT.
 - *Follow-up:* "How does Snowflake store VARIANT internally, and what's the performance implication of querying deeply nested JSON often?" → VARIANT is stored as a form of optimized columnar structure; flattening frequently-accessed JSON into relational columns during Bronze→Silver is a best practice to avoid repeated FLATTEN cost.
- 6. CASE-based classification tagging** — data classification logic based on `data_domain`.

Likely additional SQL asked live (be ready — common in Snowflake interviews):

- Write a query to find the 2nd highest salary / Nth record per group.
- QUALIFY clause usage (Snowflake-specific — simplifies the ROW_NUMBER + WHERE rn=1 pattern):

```
sql
SELECT * FROM bronze.raw_products
QUALIFY ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY load_timestamp DESC) = 1;
```

This is a strong one to mention — shows Snowflake-specific expertise, not just generic ANSI SQL.

8. Core Snowflake Architecture Questions (expect these regardless of your project notes)

Be ready — these are near-guaranteed in a Senior Snowflake role, and your notes don't cover them, so review beforehand:

- 1. Explain Snowflake's architecture** (storage layer, compute/virtual warehouse layer, cloud services layer).
- 2. Micro-partitions and clustering** — how Snowflake prunes data, when you'd define a clustering key.

3. **Time Travel & Fail-safe** — retention periods, use cases (recovering dropped table, querying historical data), difference between the two.
4. **Zero-Copy Cloning** — how you'd use it to clone PROD to a dev/test environment cheaply, especially relevant for your UAT/QA promotion flow.
5. **Virtual Warehouse sizing & scaling** — difference between scaling up (bigger warehouse) vs scaling out (multi-cluster warehouse), and how you'd approach **cost optimization** — auto-suspend/auto-resume, right-sizing warehouses per workload (ingestion vs BI query vs DBT transform).
6. **Streams & Tasks** — Snowflake-native CDC alternative to your framework's merge-based delta approach; be ready to compare/contrast if asked "have you used Streams and Tasks instead of Airflow-driven merges?"
7. **Query performance troubleshooting** — how you'd read a Query Profile, identify spillage to local/remote disk, and resolve it (warehouse resize, clustering, query rewrite).

9. Agile / Behavioral Questions

Q1: Describe your day-to-day in an Agile team. Daily standup → blockers raised to Scrum Master → sprint planning (story assignment/estimation) → retrospective (feedback/improvement).

Q2 (STAR-based — prepare a real story): "Tell me about a time you handled a production issue under pressure." Use the CR/hypercare context — a real pipeline failure during the 5–7 day monitoring window, root cause, fix, and how you communicated to Ops/client.

Q3: "Tell me about a time you disagreed with a client/data owner requirement." Use the UAT sign-off / Data Owner approval process as context — a time requirements changed post-sign-off and how you managed scope/communication.

Q4: "How do you mentor or support junior engineers?" (Expected at 12 YOE / Senior level) Have a concrete example ready — reviewing PRs, walking a junior through the framework, KT sessions to Ops.

10. Questions Abdul Should Ask the Interviewer

- What does the current Snowflake architecture/warehouse strategy look like on their side (single account, multi-account, Snowgrid/replication)?
- Do they use DBT Cloud or DBT Core, and how is CI/CD structured for DBT specifically?
- What's the team structure — dedicated platform/DevOps team vs data engineers owning CI/CD end-to-end?
- What does "senior" look like in this role — architecture ownership, mentoring, client-facing design discussions?

Gaps to Close Before the Real Interview

Your notes are strong on **process and project context** but light on a few things interviewers will probe — fill these in with your real experience beforehand:

1. Exact CI/CD tooling used (Jenkins/GitHub Actions/Azure DevOps/GitLab CI) and Git platform.
2. Any Snowflake **cost optimization** work you've personally done (warehouse right-sizing, query tuning, resource monitors).
3. Whether you've used **Streams & Tasks, Snowpipe**, or only the custom framework — be honest but bridge to what you *do* know.
4. Approximate **data volumes** (rows/day, GB/TB scale) — interviewers love a number.
5. One specific **performance problem you personally solved** (slow query, warehouse cost spike, data quality issue) with a clear before/after.